

Kubernetes

Setup Jumpbox

Install Command Line Utilities

```
1 {
2   apt-get update
3   apt-get -y install wget curl vim openssl git
4 }
```

Download Binaries

```
1 cat downloads-$(dpkg --print-architecture).txt
```

Download the binaries into a directory called `downloads` using the `wget` command:

```
1 wget -q --show-progress \
2   --https-only \
3   --timestamping \
4   -P downloads \
5   -i downloads-$(dpkg --print-architecture).txt
```

Extract the component binaries from the release archives and organize them under the `downloads` directory.

```
1 {
2   ARCH=$(dpkg --print-architecture)
3   mkdir -p downloads/{client,cni-plugins,controller,worker}
4   tar -xvf downloads/crictl-v1.32.0-linux-${ARCH}.tar.gz \
5     -C downloads/worker/
6   tar -xvf downloads/containerd-2.1.0-beta.0-linux-${ARCH}.tar.gz \
7     --strip-components 1 \
8     -C downloads/worker/
9   tar -xvf downloads/cni-plugins-linux-${ARCH}-v1.6.2.tgz \
10    -C downloads/cni-plugins/
11  tar -xvf downloads/etcd-v3.6.0-rc.3-linux-${ARCH}.tar.gz \
12    -C downloads/ \
13    --strip-components 1 \
14    etcd-v3.6.0-rc.3-linux-${ARCH}/etcdctl \
15    etcd-v3.6.0-rc.3-linux-${ARCH}/etcd
16  mv downloads/{etcdctl,kubectrl} downloads/client/
17  mv downloads/{etcd,kube-apiserver,kube-controller-manager,kube-
18    scheduler} \
19    downloads/controller/
20  mv downloads/{kubelet,kube-proxy} downloads/worker/
21  mv downloads/runc.${ARCH} downloads/worker/runc
22 }
```

Make the binaries executable.

```
1 {
2   chmod +x downloads/{client,cni-plugins,controller,worker}/*
3 }
```

Install kubectl

Use the `chmod` command to make the `kubectrl` binary executable and move it to the `/usr/local/bin/` directory:

```
1 {
2   cp downloads/client/kubectrl /usr/local/bin/
3 }
```

Result:

```
1 kubectrl version --client
```

```
1 Client Version: v1.32.3
```

```
2 Kustomize Version: v5.5.0
```

Provisioning Compute

create a `machines.txt` file with the details for the three machines

```
1 48.211.168.152 server.kubernetes.local server
2 20.246.90.103 node-0.kubernetes.local node-0 10.200.0.0/24
3 20.62.11.95 node-1.kubernetes.local node-1 10.200.1.0/24
```

Enable root SSH Access

Edit the `/etc/ssh/sshd_config` SSH daemon configuration file and set the `PermitRootLogin` option to `yes` :

```
1 sed -i \
2 's/^#*PermitRootLogin.*/PermitRootLogin yes/' \
3 /etc/ssh/sshd_config
```

Restart the `sshd` SSH server to pick up the updated configuration file:

```
1 systemctl restart sshd
```

Generate and Distribute SSH Keys

Generate and distribute an SSH keypair to the `server`, `node-0`, and `node-1`, machines, which will be used to run commands on those machines

Generate a new SSH key:

```
1 ssh-keygen

1 Generating public/private rsa key pair.
2 Enter file in which to save the key (/root/.ssh/id_rsa):
3 Enter passphrase (empty for no passphrase):
4 Enter same passphrase again:
5 Your identification has been saved in /root/.ssh/id_rsa
6 Your public key has been saved in /root/.ssh/id_rsa.pub
7
```

Copy the SSH public key to each machine:

```
1 while read IP FQDN HOST SUBNET; do
2     ssh-copy-id root@${IP}
3 done < machines.txt
```

Once each key is added, verify SSH public key access is working:

```
1 while read IP FQDN HOST SUBNET; do
2     ssh -n root@${IP} hostname
3 done < machines.txt
```

```
1 server
2 node-0
3 node-1
```

Set the hostname on each machine listed in the `machines.txt` file:

```
1 while read IP FQDN HOST SUBNET; do
2     CMD="sed -i 's/^127.0.1.1.*/127.0.1.1\t${FQDN} ${HOST}/'
/etc/hosts"
3     ssh -n root@${IP} "$CMD"
4     ssh -n root@${IP} hostnamectl set-hostname ${HOST}
5     ssh -n root@${IP} systemctl restart systemd-hostnamed
6 done < machines.txt
```

Verify the hostname is set on each machine:

```
1 while read IP FQDN HOST SUBNET; do
2     ssh -n root@${IP} hostname --fqdn
3 done < machines.txt
```

```
1 server.kubernetes.local
```

```
2 node-0.kubernetes.local
3 node-1.kubernetes.local
```

Adding `/etc/hosts` Entries To A Local Machine

Append the DNS entries from `hosts` to `/etc/hosts` :

```
1 cat hosts >> /etc/hosts
```

Adding `/etc/hosts` Entries To The Remote Machines

Copy the `hosts` file to each machine and append the contents to `/etc/hosts` :

```
1 while read IP FQDN HOST SUBNET; do
2   scp hosts root@${HOST}:/
3   ssh -n \
4     root@${HOST} "cat hosts >> /etc/hosts"
5 done < machines.txt
```

Provisioning a CA and Generating TLS Certificates

Generate the CA configuration file, certificate, and private key:

```
1 {
2   openssl genrsa -out ca.key 4096
3   openssl req -x509 -new -sha512 -noenc \
4     -key ca.key -days 3653 \
5     -config ca.conf \
6     -out ca.crt
7 }
```

Results:

```
1 ca.crt ca.key
```

Create Client and Server Certificates

Generate the certificates and private keys:

```
1 certs=(
2   "admin" "node-0" "node-1"
3   "kube-proxy" "kube-scheduler"
4   "kube-controller-manager"
5   "kube-api-server"
6   "service-accounts"
7 )

1 for i in ${certs[*]}; do
2   openssl genrsa -out "${i}.key" 4096
3
4   openssl req -new -key "${i}.key" -sha256 \
5     -config "ca.conf" -section ${i} \
6     -out "${i}.csr"
7
8   openssl x509 -req -days 3653 -in "${i}.csr" \
9     -copy_extensions copyall \
10    -sha256 -CA "ca.crt" \
11    -CAkey "ca.key" \
12    -CAcreateserial \
13    -out "${i}.crt"
14 done
```

Distribute the Client and Server Certificates

Copy the appropriate certificates and private keys to the `node-0` and `node-1` machines:

```
1 for host in node-0 node-1; do
2   ssh root@${host} mkdir /var/lib/kubelet/
```

```

3
4   scp ca.crt root@${host}:/var/lib/kubelet/
5
6   scp ${host}.crt \
7       root@${host}:/var/lib/kubelet/kubelet.crt
8
9   scp ${host}.key \
10      root@${host}:/var/lib/kubelet/kubelet.key
11 done

```

Copy the appropriate certificates and private keys to the `server` machine:

```

1 scp \
2   ca.key ca.crt \
3   kube-api-server.key kube-api-server.crt \
4   service-accounts.key service-accounts.crt \
5   root@server:~/

```

Generating Kubernetes Configuration Files for Authentication

Generate a kubeconfig file for the `node-0` and `node-1` worker nodes:

```

1 for host in node-0 node-1; do
2   kubectl config set-cluster kubernetes-the-hard-way \
3       --certificate-authority=ca.crt \
4       --embed-certs=true \
5       --server=https://server.kubernetes.local:6443 \
6       --kubeconfig=${host}.kubeconfig
7
8   kubectl config set-credentials system:node:${host} \
9       --client-certificate=${host}.crt \
10      --client-key=${host}.key \
11      --embed-certs=true \
12      --kubeconfig=${host}.kubeconfig
13
14  kubectl config set-context default \
15      --cluster=kubernetes-the-hard-way \
16      --user=system:node:${host} \
17      --kubeconfig=${host}.kubeconfig
18
19  kubectl config use-context default \
20      --kubeconfig=${host}.kubeconfig
21 done

```

Results:

```

1 node-0.kubeconfig
2 node-1.kubeconfig

```

The kube-proxy Kubernetes Configuration File

Generate a kubeconfig file for the `kube-proxy` service:

```

1 {
2   kubectl config set-cluster kubernetes-the-hard-way \
3       --certificate-authority=ca.crt \
4       --embed-certs=true \
5       --server=https://server.kubernetes.local:6443 \
6       --kubeconfig=kube-proxy.kubeconfig
7
8   kubectl config set-credentials system:kube-proxy \
9       --client-certificate=kube-proxy.crt \
10      --client-key=kube-proxy.key \
11      --embed-certs=true \
12      --kubeconfig=kube-proxy.kubeconfig
13
14  kubectl config set-context default \
15      --cluster=kubernetes-the-hard-way \
16      --user=system:kube-proxy \
17      --kubeconfig=kube-proxy.kubeconfig

```

```
18
19 kubectl config use-context default \
20   --kubeconfig=kube-proxy.kubeconfig
21 }
```

Results:

```
1 kube-proxy.kubeconfig
```

The kube-controller-manager Kubernetes Configuration File

Generate a kubeconfig file for the `kube-controller-manager` service:

```
1 {
2   kubectl config set-cluster kubernetes-the-hard-way \
3     --certificate-authority=ca.crt \
4     --embed-certs=true \
5     --server=https://server.kubernetes.local:6443 \
6     --kubeconfig=kube-controller-manager.kubeconfig
7
8   kubectl config set-credentials system:kube-controller-manager \
9     --client-certificate=kube-controller-manager.crt \
10    --client-key=kube-controller-manager.key \
11    --embed-certs=true \
12    --kubeconfig=kube-controller-manager.kubeconfig
13
14   kubectl config set-context default \
15     --cluster=kubernetes-the-hard-way \
16     --user=system:kube-controller-manager \
17     --kubeconfig=kube-controller-manager.kubeconfig
18
19   kubectl config use-context default \
20     --kubeconfig=kube-controller-manager.kubeconfig
21 }
```

Results:

```
1 kube-controller-manager.kubeconfig
2
```

The kube-scheduler Kubernetes Configuration File

Generate a kubeconfig file for the `kube-scheduler` service:

```
1 {
2   kubectl config set-cluster kubernetes-the-hard-way \
3     --certificate-authority=ca.crt \
4     --embed-certs=true \
5     --server=https://server.kubernetes.local:6443 \
6     --kubeconfig=kube-scheduler.kubeconfig
7
8   kubectl config set-credentials system:kube-scheduler \
9     --client-certificate=kube-scheduler.crt \
10    --client-key=kube-scheduler.key \
11    --embed-certs=true \
12    --kubeconfig=kube-scheduler.kubeconfig
13
14   kubectl config set-context default \
15     --cluster=kubernetes-the-hard-way \
16     --user=system:kube-scheduler \
17     --kubeconfig=kube-scheduler.kubeconfig
18
19   kubectl config use-context default \
20     --kubeconfig=kube-scheduler.kubeconfig
21 }
```

Results:

```
1 kube-scheduler.kubeconfig
```

The admin Kubernetes Configuration File

Generate a kubeconfig file for the `admin` user:

```
1 {
2   kubectl config set-cluster kubernetes-the-hard-way \
3     --certificate-authority=ca.crt \
4     --embed-certs=true \
5     --server=https://127.0.0.1:6443 \
6     --kubeconfig=admin.kubeconfig
7
8   kubectl config set-credentials admin \
9     --client-certificate=admin.crt \
10    --client-key=admin.key \
11    --embed-certs=true \
12    --kubeconfig=admin.kubeconfig
13
14   kubectl config set-context default \
15     --cluster=kubernetes-the-hard-way \
16     --user=admin \
17     --kubeconfig=admin.kubeconfig
18
19   kubectl config use-context default \
20     --kubeconfig=admin.kubeconfig
21 }
```

Results:

```
1 admin.kubeconfig
```

Distribute the Kubernetes Configuration Files

Copy the `kubelet` and `kube-proxy` kubeconfig files to the `node-0` and `node-1` machines:

```
1 for host in node-0 node-1; do
2   ssh root@${host} "mkdir -p /var/lib/{kube-proxy,kubelet}"
3
4   scp kube-proxy.kubeconfig \
5     root@${host}:/var/lib/kube-proxy/kubeconfig \
6
7   scp ${host}.kubeconfig \
8     root@${host}:/var/lib/kubelet/kubeconfig
9 done
```

Copy the `kube-controller-manager` and `kube-scheduler` kubeconfig files to the `server` machine:

```
1 scp admin.kubeconfig \
2   kube-controller-manager.kubeconfig \
3   kube-scheduler.kubeconfig \
4   root@server:~/
```

Generating the Data Encryption Config and Key

The Encryption Key

Generate an encryption key:

```
1 export ENCRYPTION_KEY=$(head -c 32 /dev/urandom | base64)
```

The Encryption Config File

Create the `encryption-config.yaml` encryption config file:

```
1 envsubst < configs/encryption-config.yaml \
2   > encryption-config.yaml
```

Copy the `encryption-config.yaml` encryption config file to each controller instance:

```
1 scp encryption-config.yaml root@server:~/
```

Bootstrapping the etcd Cluster

Copy `etcd` binaries and systemd unit files to the `server` machine:

```
1 scp \  
2   downloads/controller/etcd \  
3   downloads/client/etcdctl \  
4   units/etcd.service \  
5   root@server:~/
```

Install the etcd Binaries

Extract and install the `etcd` server and the `etcdctl` command line utility:

```
1 {  
2   mv etcd etcdctl /usr/local/bin/  
3 }
```

Configure the etcd Server

```
1 {  
2   mkdir -p /etc/etcd /var/lib/etcd  
3   chmod 700 /var/lib/etcd  
4   cp ca.crt kube-api-server.key kube-api-server.crt \  
5     /etc/etcd/  
6 }
```

Create the `etcd.service` systemd unit file:

```
1 mv etcd.service /etc/systemd/system/
```

Start the etcd Server

```
1 {  
2   systemctl daemon-reload  
3   systemctl enable etcd  
4   systemctl start etcd  
5 }
```

Bootstrapping the Kubernetes Control Plane

Connect to the `jumpbox` and copy Kubernetes binaries and systemd unit files to the `server` machine:

```
1 scp \  
2   downloads/controller/kube-apiserver \  
3   downloads/controller/kube-controller-manager \  
4   downloads/controller/kube-scheduler \  
5   downloads/client/kubectl \  
6   units/kube-apiserver.service \  
7   units/kube-controller-manager.service \  
8   units/kube-scheduler.service \  
9   configs/kube-scheduler.yaml \  
10  configs/kube-apiserver-to-kubelet.yaml \  
11  root@server:~/
```

Provision the Kubernetes Control Plane

Create the Kubernetes configuration directory:

```
1 mkdir -p /etc/kubernetes/config
```

Install the Kubernetes Controller Binaries

Install the Kubernetes binaries:

```
1 {  
2   mv kube-apiserver \  
3     kube-controller-manager \  
4     kube-scheduler kubectl \  
5 }
```

```
5 /usr/local/bin/
6 }
```

Configure the Kubernetes API Server

```
1 {
2   mkdir -p /var/lib/kubernetes/
3
4   mv ca.crt ca.key \
5     kube-api-server.key kube-api-server.crt \
6     service-accounts.key service-accounts.crt \
7     encryption-config.yaml \
8     /var/lib/kubernetes/
9 }
```

Create the `kube-apiserver.service` systemd unit file:

```
1 mv kube-apiserver.service \
2   /etc/systemd/system/kube-apiserver.service
```

Configure the Kubernetes Controller Manager

Move the `kube-controller-manager.kubeconfig` into place:

```
1 mv kube-controller-manager.kubeconfig /var/lib/kubernetes/
```

Create the `kube-controller-manager.service` systemd unit file:

```
1 mv kube-controller-manager.service /etc/systemd/system/
```

Configure the Kubernetes Scheduler

Move the `kube-scheduler.kubeconfig` into place:

```
1 mv kube-scheduler.kubeconfig /var/lib/kubernetes/
```

Create the `kube-scheduler.yaml` configuration file:

```
1 mv kube-scheduler.yaml /etc/kubernetes/config/
```

Create the `kube-scheduler.service` systemd unit file:

```
1 mv kube-scheduler.service /etc/systemd/system/
```

Start the Controller Services

```
1 {
2   systemctl daemon-reload
3
4   systemctl enable kube-apiserver \
5     kube-controller-manager kube-scheduler
6
7   systemctl start kube-apiserver \
8     kube-controller-manager kube-scheduler
9 }
```

Verification

Verify this using the `kubectl` command line tool:

```
1 kubectl cluster-info \
2   --kubeconfig admin.kubeconfig
```

```
1 Kubernetes control plane is running at https://127.0.0.1:6443
```

Create the `system:kube-apiserver-to-kubelet` [ClusterRole](#) with permissions to access the Kubelet API and perform most common tasks associated with managing pods:

```
1 kubectl apply -f kube-apiserver-to-kubelet.yaml \
```

```
2 --kubeconfig admin.kubeconfig
```

Bootstrapping the Kubernetes Worker Nodes

The commands in this section must be run from the `jumpbox`.

Copy the Kubernetes binaries and systemd unit files to each worker instance:

```
1 for HOST in node-0 node-1; do
2     SUBNET=$(grep ${HOST} machines.txt | cut -d " " -f 4)
3     sed "s|SUBNET|${SUBNET}|g" \
4         configs/10-bridge.conf > 10-bridge.conf
5
6     sed "s|SUBNET|${SUBNET}|g" \
7         configs/kubelet-config.yaml > kubelet-config.yaml
8
9     scp 10-bridge.conf kubelet-config.yaml \
10        root@${HOST}:~/
11 done
```

```
1 for HOST in node-0 node-1; do
2     scp \
3         downloads/worker/* \
4         downloads/client/kubectl \
5         configs/99-loopback.conf \
6         configs/containerd-config.toml \
7         configs/kube-proxy-config.yaml \
8         units/containerd.service \
9         units/kubelet.service \
10        units/kube-proxy.service \
11        root@${HOST}:~/
12 done
```

```
1 for HOST in node-0 node-1; do
2     scp \
3         downloads/cni-plugins/* \
4         root@${HOST}:~/cni-plugins/
5 done
```

Provisioning a Kubernetes Worker Node

Install the OS dependencies:

```
1 {
2     apt-get update
3     apt-get -y install socat conntrack ipset kmod
4 }
```

Create the installation directories:

```
1 mkdir -p \
2     /etc/cni/net.d \
3     /opt/cni/bin \
4     /var/lib/kubelet \
5     /var/lib/kube-proxy \
6     /var/lib/kubernetes \
7     /var/run/kubernetes
```

Install the worker binaries:

```
1 {
2     mv crictl kube-proxy kubelet runc \
3         /usr/local/bin/
4     mv containerd containerd-shim-runc-v2 containerd-stress /bin/
5     mv cni-plugins/* /opt/cni/bin/
6 }
```

Configure CNI Networking

Create the `bridge` network configuration file:

```
1 mv 10-bridge.conf 99-loopback.conf /etc/cni/net.d/
```

To ensure network traffic crossing the CNI `bridge` network is processed by `iptables`, load and configure the `br-netfilter` kernel module:

```
1 {
2   modprobe br-netfilter
3   echo "br-netfilter" >> /etc/modules-load.d/modules.conf
4 }
```

```
1 {
2   echo "net.bridge.bridge-nf-call-iptables = 1" \
3     >> /etc/sysctl.d/kubernetes.conf
4   echo "net.bridge.bridge-nf-call-ip6tables = 1" \
5     >> /etc/sysctl.d/kubernetes.conf
6   sysctl -p /etc/sysctl.d/kubernetes.conf
7 }
```

Configure containerd

Install the `containerd` configuration files:

```
1 {
2   mkdir -p /etc/containerd/
3   mv containerd-config.toml /etc/containerd/config.toml
4   mv containerd.service /etc/systemd/system/
5 }
```

Configure the Kubelet

Create the `kubelet-config.yaml` configuration file:

```
1 {
2   mv kubelet-config.yaml /var/lib/kubelet/
3   mv kubelet.service /etc/systemd/system/
4 }
```

Configure the Kubernetes Proxy

```
1 {
2   mv kube-proxy-config.yaml /var/lib/kube-proxy/
3   mv kube-proxy.service /etc/systemd/system/
4 }
```

Start the Worker Services

```
1 {
2   systemctl daemon-reload
3   systemctl enable containerd kubelet kube-proxy
4   systemctl start containerd kubelet kube-proxy
5 }
```

Configuring kubectl for Remote Access

Generate a kubeconfig file suitable for authenticating as the `admin` user:

```
1 {
2   kubectl config set-cluster kubernetes-the-hard-way \
3     --certificate-authority=ca.crt \
4     --embed-certs=true \
5     --server=https://server.kubernetes.local:6443
6
7   kubectl config set-credentials admin \
8     --client-certificate=admin.crt \
9     --client-key=admin.key
10 }
```

```
11 kubectl config set-context kubernetes-the-hard-way \  
12   --cluster=kubernetes-the-hard-way \  
13   --user=admin  
14  
15 kubectl config use-context kubernetes-the-hard-way  
16 }
```

Provisioning Pod Network Routes

Print the internal IP address and Pod CIDR range for each worker instance:

```
1 {  
2   SERVER_IP=$(grep server machines.txt | cut -d " " -f 1)  
3   NODE_0_IP=$(grep node-0 machines.txt | cut -d " " -f 1)  
4   NODE_0_SUBNET=$(grep node-0 machines.txt | cut -d " " -f 4)  
5   NODE_1_IP=$(grep node-1 machines.txt | cut -d " " -f 1)  
6   NODE_1_SUBNET=$(grep node-1 machines.txt | cut -d " " -f 4)  
7 }
```

```
1 ssh root@server <<EOF  
2   ip route add ${NODE_0_SUBNET} via ${NODE_0_IP}  
3   ip route add ${NODE_1_SUBNET} via ${NODE_1_IP}  
4 EOF
```

```
1 ssh root@node-0 <<EOF  
2   ip route add ${NODE_1_SUBNET} via ${NODE_1_IP}  
3 EOF
```

```
1 ssh root@node-1 <<EOF  
2   ip route add ${NODE_0_SUBNET} via ${NODE_0_IP}  
3 EOF
```